

RISC-V os update 1

James Ryan, Evan Rosenfeld, Ant Nosaryev
Spring 2026

Reflection

- Got a simple kernel running in QEMU
- No progress on Ox64
 - Poor / nonexistent JTAG support
 - Limited documentation
- Studied a couple historical kernels
 - Linux
 - Plan 9 from Bell Labs
- We really underestimated how complicated getting the board up could be...

Winter research: Baochip-1x and the Xous Kernel

- **Issue:** ARM is a closed architecture, and there is a large price disparity between the Cortex-M and Cortex-A chips
 - Key difference is the presence of a memory management unit (MMU) in Cortex-A
 - ARM intentionally omit the MMU from Cortex-M chips to force applications that rely on the it (e.g. anything with a Linux kernel) to pay more for Cortex-A chips or rely on ARM's proprietary memory protection unit (MPU).
- **Goal (for Baochip):** develop a low cost chip with an MMU and modern I/O
 - Created by Andrew Huang (<https://baochip.com/>)
 - Partially open RTL <https://github.com/baochip/baochip-1x?tab=readme-ov-file>
 - RISC-V based
 - Programmable I/O, similar to the Raspberry Pi Pico PIO
 - Allows for generic IO protocol emulation

Talk: <https://media.ccc.de/v/39c3-xous-a-pure-rust-rethink-of-the-embedded-operating-system#t=793> (Dec. 2025)

About 40 minutes long.

Winter research: Baochip-1x and the Xous Kernel

- **Xous microkernel** pairs with the Baochip, using rules from the Rust language borrow checker to ensure memory security:
 - Memory is mutable (able to be changed) xor shared
 - Memory at any point has one owner only
 - Microkernel design: the kernel is very small, and handles passing messages between processes. Otherwise, each process is isolated in memory

<https://github.com/betrusted-io/xous-core>

Winter research: Baochip-1x and the Xous Kernel

In the Xous operating system, memory can be:

- Moved: *a process gives up control of memory to another process, transferring ownership*
- Borrowed (shared): *the same piece of memory is accessed by multiple processes, and it cannot be changed.*
- Mutably borrowed: *a piece of memory is temporarily owned by another process, and will be returned. There is only one active pointer which can change that memory's contents*

We could be a little inaccurate

See <https://users.rust-lang.org/t/rust-mutability-moving-and-borrowing-the-straight-dope/22166>

Winter research: Baochip-1x and the Xous Kernel

Xous:

- Detaches memory from a process and passes it to another process, and is able to use Rust's built in borrow checking to ensure safety
 - Interaction between processes is just passing memory pages around, handled by the MMU
- Rest of the kernel is user-space programs
 - Hardware (GPIO, hardware timer) are addressed via memory
 - User space process can be given ownership of the timer, for example, and be responsible for scheduling programs.
- Built for security focused IoT

Further reading

<https://baochip.github.io/baochip-1x/ch00-00-introduction.html>

<https://betrusted.io/xous-book/>

Current work from this semester

- Asked Andrew nicely for a chip, he agreed to ship us a handful
- Ordered a board from JLCPCB to attach these onto
 - Will ask PCB Lab to help attach chips to board.
- In the meantime, there's desktop tools we can use to write programs for it



pictured

Goals for the semester

- This seems really interesting to us!
- Get the desktop tools running to write a basic program for this OS, and then inspect it running physically
 - Chips are not publicly available yet, so we need to put our own dev board together
- Try to get the generic I/O subsystem to do something interesting.

Goals for the semester?